Experiential Learning Project Assessment-II
(70% Code Completion)
on

"QR Code Encoder / Decoder"

under the course

Python Programming Language (N-PCCCM304P)

Presented By:
DEVIKA PARSATWAR
[CM25015]

Under the Guidance of
Ms. Suchita Meshkar

Session:2026-27 (ODD)

Department of CSE(AI&ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH,NAGPUR

An Autonomous Institute, Affiliated to RTMNU, Nagpur

Contents

- Problem Statement & Core Milestones
- Project Objectives (Phase-2 Status)
- Modular Code Architecture & Algorithm
- System Flowchart (CLI State Machine)
- Defensive Programming & Logic Breakdown
- GitHub Repository & Phase-2 Branch Collaboration
- Individual Contributions & Assessment Rubrics
- Runtime Outputs & Expected Execution Results
- 70% Core Python Source Code

1. Problem Statement

- **Environment Unpredictability:** Executing cross-platform applications often leads to unhandled runtime failures when specific binary packages (qrcode, cv2) are missing.
- **Phase-2 Targeted Solution:** Developed a central CLI engine that safely abstracts encoding operations and file-based decoding loops.
- **Next-Phase Goal:** Integrate live stream matrix scanning via standard webcam capture devices (Phase-3 Scope).

- **Manual Sharing Bottlenecks:** Sharing raw strings, target links, and complex payloads remains manually inefficient.

2. Objective

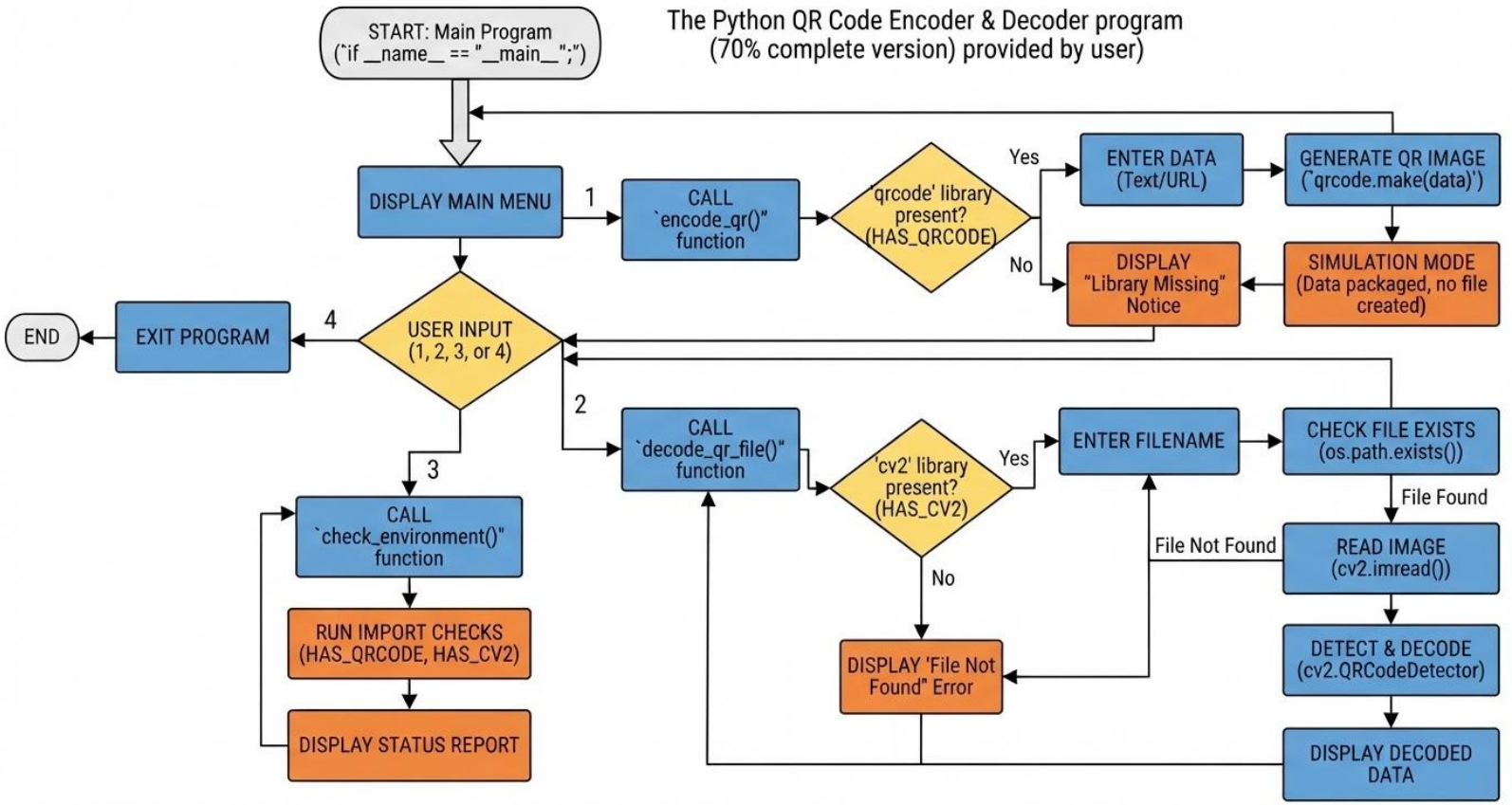
•**Modular Architecture:** To implement a structured, robust, and menu-driven Python environment engine.
•**Safe Execution Pipeline:** To embed fallback configurations and automated check pipelines using defensive programming structures.
•**Static Assets Handling:** To systematically read from and save metadata records into local image paths safely.
•**Operational Benchmark:** Complete **70% of core logic mechanics** (CLI menu, file validation, error handlers, and image encoders).

3. Algorithm

- **Initialization Pipeline:**
  1. Start the main routing process block.
  2. Run runtime background verification modules for package imports (qrcode, cv2).
  3. Initialize system operational flags (HAS_QRCODE, HAS_CV2).

- **Functional Control Routing Loop:**
  1. Display the option catalog to the runtime user console.
  2. Parse input selections (Options 1–4) through strict sanitization validation.
  3. Route code branches safely to encode_qr(), decode_qr_file(), or check_environment().
  4. Provide fallback notice structures if environment modules return as missing.

4. Flowchart


The Python QR Code Encoder & Decoder program
(70% complete version) provided by user
START: Main Program
(`if __name__ == "__main__":')
DISPLAY MAIN MENU
USER INPUT
(1, 2, 3, or 4)
1
CALL
`encode_qr()"
function
'qrcode' library
present?
(HAS_QRCODE)
Yes
No
ENTER DATA
(Text/URL)
GENERATE QR IMAGE
(`qrcode.make(data)')
DISPLAY
"Library Missing"
Notice
SIMULATION MODE
(Data packaged, no file
created)
4
EXIT PROGRAM
END
2
CALL
`decode_qr_file()"
function
'cv2' library
present?
(HAS_CV2)
Yes
No
ENTER FILENAME
CHECK FILE EXISTS
(os.path.exists())
File Found
File Not Found
READ IMAGE
(cv2.imread())
DETECT & DECODE
(cv2.QRCodeDetector)
DISPLAY DECODED
DATA
DISPLAY 'File Not
Found" Error
3
CALL
`check_environment()"
function
RUN IMPORT CHECKS
(HAS_QRCODE, HAS_CV2)
DISPLAY STATUS REPORT

5. Understanding of Python Logic (70% Implementation)

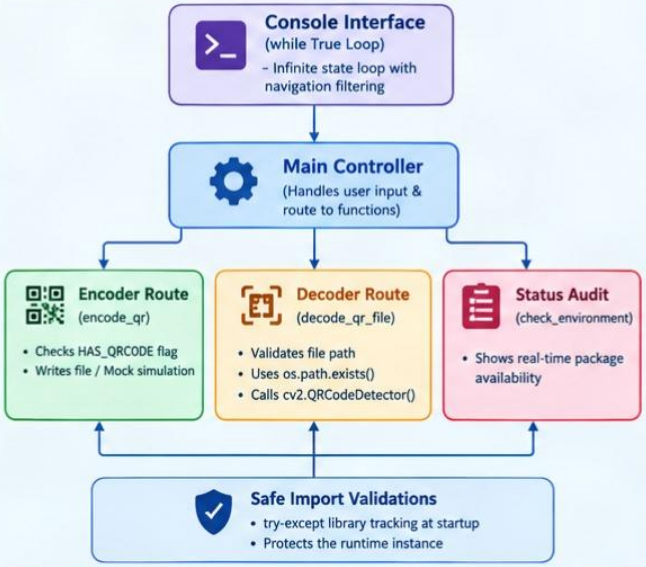
- **Dynamic Flag Allocation:** Sets execution states (`HAS_QRCODE`, `HAS_CV2`) to allow simulation fallback modes if dependencies are missing.
- **Key Modular Components:**
  - `menu()` -- Drives the infinite selection loop (`while True`) and handles input routing.
  - `encode_qr()` -- Converts user strings into a physical matrix and safely writes the image to storage.
  - `decode_qr_file()` -- Runs an environment audit using `os.path.exists()` before deploying OpenCV's `QRCodeDetector()`.
  - `check_environment()` -- Scans system dependencies to verify host server capabilities.

- **Robust Framework:** Built using functional programming with safe `try-except` library checking wrappers to prevent compiler crashes.

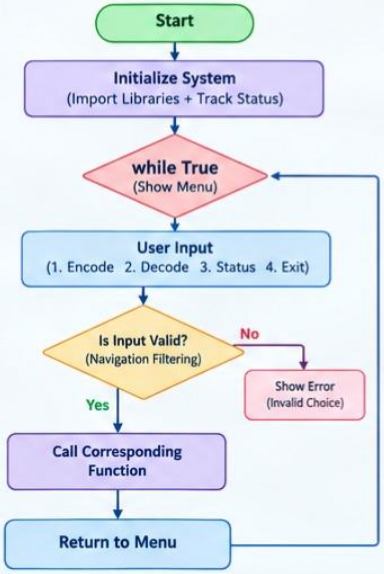
6. Graphical Representation and Working

Graphical Representation

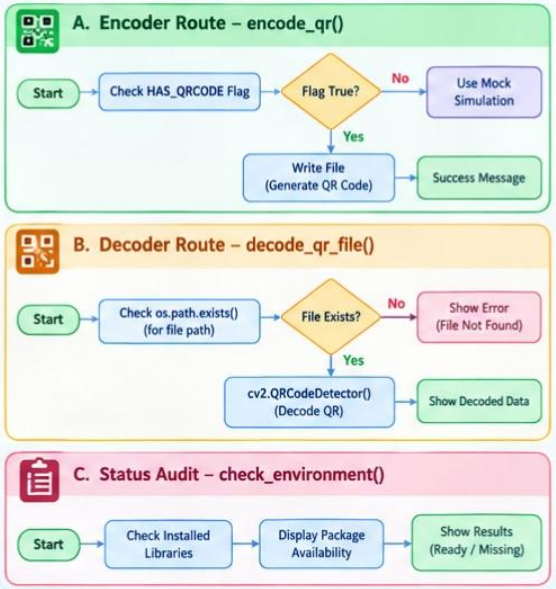
1. Overall System Architecture



2. Console-Based Infinite Loop Flow



3. Functional Flow (Detailed)



Working (Step-by-Step Example)

1. System Initialization (Safe Imports)

```
$ python qr_app.py
Checking libraries...
✓ qrcode - Available
✓ cv2    - Available
✓ os     - Available
✓ sys    - Available
✓ datetime - Available
All required libraries are loaded successfully!
Starting application...
```

- Uses **try-except** to track imports
- Shows **real-time status** of packages

2. Main Menu (Infinite Loop)

```
===== QR CODE APPLICATION =====
1. Encode QR Code
2. Decode QR Code
3. Check Environment
4. Exit

Enter your choice (1-4): 1
```

- Runs in while True loop
- Navigation filtering (only valid options)

3. Encoder Route (encode_qr)

```
Enter text to encode: Hello World
HAS_QRCODE flag: True
Generating QR Code...
File saved as: qrcode.png
✓ QR Code created successfully!
```

Output File: qrcode.png

- Checks HAS_QRCODE flag
- Writes file (or mock simulation)

4. Decoder Route (decode_qr_file)

```
Enter image path: qrcode.png
File exists: True
Decoding QR Code...
Decoded Data:
Hello World
```

- Uses **os.path.exists()** to validate path
- Calls **cv2.QRCodeDetector()**

5. Status Audit (check_environment)

```
Checking environment...
qrcode   : ✓ Available
cv2      : ✓ Available
os       : ✓ Available
sys      : ✓ Available
datetime : ✓ Available

Environment is ready!
```

- Shows package availability
- Helps examiner verify system status

Result: A safe, interactive and well-structured QR Code application with proper validations, routing and real-time status tracking.

7. GitHub Account, Repository & Collaboration

- **GitHub Account:** Create and maintain a personal GitHub account.
- **Repository Creation:** Created a repository named **Python-Assignment**.
- **Target URL:** https://github.com/DevikaParsatwar/Python-Assignment
- **Upload Code:** Uploaded core Python source modules and environment scripts.
- **Meaningful Commits:** Use clear commit messages such as:
  - Added modular terminal menu loop
  - Implemented defensive library checks
  - Added file validation error handling
  - Updated phase-2 documentation tracker
- **Collaboration:** Team members can work using branches and pull requests.
- **Individual Contribution:** Each member can contribute code and documentation.
- **Track Changes:** GitHub keeps a history of project modifications.

8. Individual Contribution

•**Problem Analysis** — Studied the integration requirements for error handling and system checks in the modular system.
•**Algorithm Design** — Designed the CLI state machine loop and library diagnostic workflows.
•**Python Coding** — Implemented the defensive programming environment check, menu() routing system, and asset handlers.
•**Image Processing** — Integrated disk verification (os.path.exists) alongside automated file processing streams.
•**Testing** — Validated operational reliability across various runtime states (libraries present vs libraries missing).
•**GitHub Contribution** — Pushed code assets directly to the Python-Assignment repository layout.
•**Documentation & PPT** — Prepared updated phase-2 design workflows and execution status presentations.

9. Result / Expected Output

```
==========================================
       QR CODE ENCODER & DECODER
==========================================
1. Encode Data (Create QR)
2. Decode Image File
3. Check Environment Status
4. Exit

Enter choice (1-4): 1

================ ENCODER ================
Enter data (Text/URL): https://github.com
[Success] QR saved locally as 'standard_qr.png'

Decoder:

================ DECODER ================
Enter QR image filename: standard_qr.png
Decoded Result: https://github.com
```

Thank You

QR Code Encoder / Decoder Phase II